

RAZORPAY BUILDATHON · TRACK 01 · AI GROWTH & AGENTIC COMMERCE

# AEGIS RAIL

Implementation Plan, Feasibility, Uniqueness &  
Agentic Architecture — Deep-Dive Companion Document

DOCUMENT 2 OF 2

---

Companion to the AEGIS RAIL Project Proposal  
Chennai Institute of Technology · AD4V38 — AI in Cyber Security

# 1. Uniqueness & Competitive Differentiation

The honest test for uniqueness isn't "has anyone built an AI agent before" — dozens of Track 01 submissions will. The real question is whether AEGIS RAIL solves a *different layer of the problem* than everyone else, in a way that's still directly useful to Razorpay. Here's how it compares.

## 1.1 Versus the likely field of Track 01 submissions

| Typical submission pattern                   | What it actually is                                  | AEGIS RAIL's position                                                                          |
|----------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| "Conversational checkout for [one merchant]" | A single-purpose chatbot wired to one demo store     | AEGIS RAIL is merchant-agnostic — one gateway, any merchant, onboarded in minutes              |
| "Upsell/cross-sell agent"                    | A recommendation engine bolted onto checkout         | Built as one interchangeable sub-agent inside AEGIS RAIL's orchestrator, not the whole product |
| "Campaign orchestrator"                      | Marketing automation dressed as agentic commerce     | Out of scope by design — AEGIS RAIL focuses on the transaction-trust problem, not marketing    |
| Agent-readable catalog demos                 | A one-off schema for a single hardcoded product list | A general Catalog Agentification Engine that ingests <i>any</i> merchant's raw data            |

## 1.2 Versus Razorpay's own Agent Studio

This is the differentiation that matters most, because Razorpay's own team will read this submission. AEGIS RAIL is deliberately **not** a competitor to Agent Studio's Abandoned Cart, Dispute Responder, Subscription Recovery, or Cashflow Forecaster agents — it is the layer *underneath* them, solving the one problem none of those products touch: standardized, verifiable, protocol-agnostic access for merchants who are not enterprise pilot partners.

## 1.3 Versus global agentic-commerce infrastructure plays

Global players like Crossmint, ATPX, and ZeroClick are building similar "protocol adapter + agent wallet" infrastructure, but for a global, crypto/stablecoin-first audience. AEGIS RAIL is deliberately narrower and deeper: India-first, UPI/NPCI-aware, built natively on Razorpay's own MCP server and test-mode APIs, and targeted specifically at the long tail of Indian SMB merchants who will never touch a stablecoin wallet. That narrowness is a feature for a Razorpay-specific buildathon, not a weakness.

**The one-sentence differentiation:** most submissions will demonstrate an agent doing a task; AEGIS RAIL demonstrates the infrastructure that lets Razorpay *trust* any agent doing any task, for any merchant — which is the actual bottleneck standing between Razorpay's pilots and Razorpay's whole platform.

## 2. Feasibility Assessment

Every layer of AEGIS RAIL was checked against one question: can this actually be built, demoed, and defended solo, inside a buildathon window, using only publicly available tools? Scored 1 (hardest) to 5 (easiest) for build feasibility.

| Layer                                  | Feasibility | Reasoning                                                                                                                                                 |
|----------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layer 1 — Protocol Translation         | 4 / 5       | ACP, AP2, and UCP all publish JSON schemas; parsing and normalizing them is a well-scoped engineering task, not a research problem                        |
| Layer 2 — Catalog Agentification       | 4 / 5       | Structured extraction from spreadsheets/product pages is a mature LLM task; MCP tool-schema generation is directly documented by Razorpay                 |
| Layer 3 — Trust & Verification Gateway | 3 / 5       | Signature verification is standard cryptography; the anomaly scorer is buildable quickly but only to a provisional accuracy level without real fraud data |
| Layer 4 — Orchestrator Agent           | 5 / 5       | This is a direct, well-supported use of the Claude Agent SDK — the same foundation Razorpay itself uses                                                   |
| Layer 5 — Razorpay Execution Layer     | 5 / 5       | The Razorpay MCP Server and test-mode APIs are public, documented, and designed exactly for this kind of integration                                      |
| Layer 6 — Audit Ledger                 | 5 / 5       | A hash-chained append-only log is a well-understood, low-risk build                                                                                       |

### 2.1 Time Feasibility

Given a solo build, effort is front-loaded onto the layers that make the demo credible (1, 2, 3) and the layers that make it functional (4, 5), while Layer 6 stays intentionally simple — a working ledger and a clean dashboard view, not a full analytics product.

### 2.2 Data Feasibility

No real agent-fraud dataset is publicly available to a student team. The plan is to generate synthetic transaction-behavior data that models known fraud patterns (velocity spikes, spend-limit creep, replayed signatures) closely enough to demonstrate the anomaly scorer meaningfully, while stating plainly in the demo that production deployment would require retraining on real transaction history.

### 2.3 Dependency Risk

The biggest external dependency is NPCI's Unified Agent Protocol, which is not public. This is fully de-risked by building a protocol-compliant simulator rather than waiting on or assuming access to the real spec — the Protocol Adapter Agent is written so the simulator can be swapped for the real UAP the moment it ships, without touching the rest of the system.

### 2.4 Overall Feasibility Verdict

AEGIS RAIL is feasible as a working, demoable prototype within a buildathon timeline, provided scope discipline is maintained: full depth on Layers 1–3, functional depth on Layers 4–6, and no attempt to claim production-grade accuracy or live third-party protocol federation.

### 3. Modules We Already Have

Nothing here starts from zero. Prior work on SENTINEL-AI and Razorpay's own public developer tooling give a real head start.

| Module                                       | Source                              | How it's reused in AEGIS RAIL                                                                                                   |
|----------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Anomaly detection pipeline                   | SENTINEL-AI                         | Retargeted from network/threat telemetry to transaction-behavior features for the Trust & Verification Gateway's anomaly scorer |
| SHAP/LIME explainability wrapper             | SENTINEL-AI                         | Directly reused to generate the plain-language reasoning behind every allow/block decision                                      |
| Zero-trust architecture patterns             | SENTINEL-AI                         | The "never trust, always verify" design of Layer 3 is a direct application of the same principle                                |
| Graph-based relationship modeling experience | SENTINEL-AI (graph neural networks) | Reusable if the cross-merchant fraud-ring extension (Section 4) is built out                                                    |
| MLOps CI/CD familiarity                      | SENTINEL-AI                         | Applied to keep the anomaly scorer versioned and testable rather than a one-off script                                          |
| Razorpay MCP Server                          | Razorpay (public)                   | The execution backbone for every real transaction in the demo                                                                   |
| Razorpay test-mode APIs & dashboard          | Razorpay (public)                   | Orders, Payment Links, Payment Pages, Settlements — all callable without needing production keys                                |
| Razorpay llms.txt                            | Razorpay (public)                   | Used to ground the Orchestrator Agent's understanding of Razorpay's API surface accurately                                      |
| Claude Agent SDK                             | Anthropic (public)                  | The same foundation Razorpay's own Agent Studio is built on — direct architectural alignment                                    |

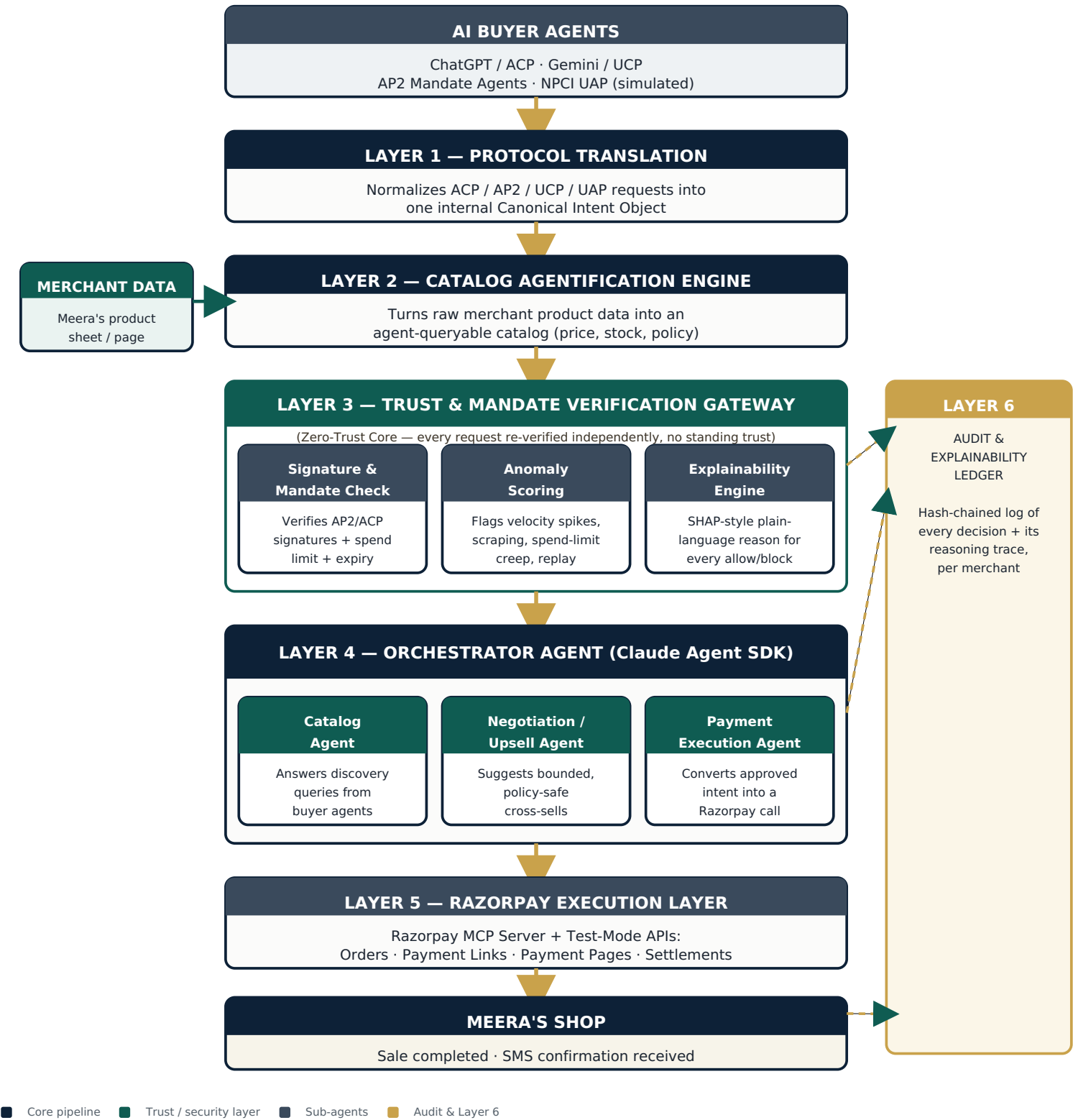
## 4. What Else We Can Implement (Extension Roadmap)

Beyond the MVP, these extensions were deliberately scoped out but designed to be addable without re-architecting the system — useful either as stretch goals if time allows, or as a "here's where this goes next" answer to a judge's question.

| Extension module                                         | What it adds                                                                                                                                         | Effort     |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Cross-merchant fraud-ring detection                      | A graph-neural-network layer that spots the same rogue agent probing multiple merchants in a coordinated pattern, not just one merchant in isolation | High       |
| Merchant "Agent-Ready" trust score & certification badge | A public reputation signal merchants can display once AEGIS RAIL has verified their catalog and transaction history are clean                        | Medium     |
| Voice/Hinglish flagged-transaction review                | Lets a merchant like Meera ask, in Hinglish, "why was that ₹4000 order blocked?" and get a spoken explanation                                        | Medium     |
| x402 stablecoin micropayment support                     | Extends the Protocol Adapter Agent to handle machine-to-machine, pay-per-call transactions for digital/API-style merchants                           | Medium     |
| Compliance evidence auto-generator                       | Turns the audit ledger into a structured report formatted for a future RBI/NPCI audit request                                                        | Low-Medium |
| B2B bulk-agent negotiation                               | Lets a procurement AI agent negotiate quantity discounts with the Negotiation Agent within pre-approved bounds                                       | High       |

## 5. Agentic Architecture Diagram

The full request-to-settlement flow, from an AI buyer agent's first request through to a completed, audited Razorpay transaction.



## 6. Implementation Plan

Structured in five phases, ordered so that a demoable, honest MVP exists as early as possible, with every later phase adding depth rather than being required for the system to work at all.

### Phase 0 — Setup & Grounding

| Tasks                                                                                                                                              | Deliverable                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Set up Razorpay test-mode account and API keys; install and test the Razorpay MCP Server locally; read the ACP, AP2, UCP public schemas end to end | Working MCP connection confirmed; protocol schemas documented in own notes |

### Phase 1 — Core Pipeline: Translation + Catalog + Trust Gateway

| Tasks                                                                                                                                                                                                                                                                  | Deliverable                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Build the Protocol Adapter Agent for at least ACP and AP2 formats; build the Catalog Agentification Engine against a sample merchant product sheet; build the Trust & Verification Gateway's signature check and a first-pass anomaly scorer trained on synthetic data | A single verified purchase intent can flow from a simulated AI buyer request through to an "approved" decision with an explanation string |

### Phase 2 — Orchestrator & Execution

| Tasks                                                                                                                                                                                                                                                               | Deliverable                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Wire the Orchestrator Agent (Claude Agent SDK) to route approved intents to the Payment Execution Agent; connect the Payment Execution Agent to the Razorpay MCP Server to create real test-mode Orders and Payment Links; build the graceful-failure fallback path | An approved AI-buyer intent produces a real Razorpay test-mode transaction end to end, with a working fallback if it fails |

### Phase 3 — Audit Ledger & Merchant Dashboard

| Tasks                                                                                                                                      | Deliverable                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Implement the hash-chained audit log; build a simple merchant-facing view showing each transaction's decision and plain-language reasoning | A judge or merchant can open the dashboard and see exactly why every transaction was allowed or blocked |

### Phase 4 — Hardening, Second Protocol, and Demo Rehearsal

| Tasks                                                                                                                                                                                                                                                  | Deliverable                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Add a second/third protocol adapter (UCP and/or simulated UAP); stress-test the anomaly scorer with deliberately adversarial synthetic requests; rehearse the full Meera/Nova demo narrative end to end; prepare the honest limitations talking points | A rehearsed, resilient live demo plus a clear, confident answer ready for any "what doesn't this do yet" question |

**Scope discipline rule:** if time runs short, cut breadth (fewer protocols, simpler catalog inputs) before cutting depth (the trust gateway's explainability must always work convincingly — it is the differentiator).

## 7. Demo Checklist & Success Criteria

| Check                     | What "done" looks like                                                                                                                       |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| End-to-end transaction    | A simulated AI buyer request results in a real Razorpay test-mode Order/Payment Link, visible in the Razorpay dashboard                      |
| Explainability            | Every decision shown in the demo has a plain-language reasoning string a non-technical judge can read and understand                         |
| Bounded & gated           | At least one live demo request is deliberately over the spend mandate and is correctly blocked, with the reason shown                        |
| Graceful failure          | At least one simulated failure (timeout, malformed mandate) is shown falling back to a standard Payment Link                                 |
| Audit trail               | The dashboard shows a real, tamper-evident log of every demo transaction with its full reasoning trace                                       |
| Merchant onboarding speed | A fresh, previously-unseen product sheet is turned into an agent-readable catalog live, in front of the judges, in under a couple of minutes |